

Project charter: open source platform for centralized AI agent governance

Sep 28, 2026 ·

Executive summary

This project builds an open source, modular, self-hosted control plane to govern all of an organization's AI agents from a single place: registry, identity, permissions, policies, resources, skills, observability, compliance and lifecycle, with a working kill switch.

It does not replace existing specialized tools. It integrates them through standard module contracts, so each capability (policy engine, MCP gateway, skills repository, compliance) can be covered by any vendor's tool, open or commercial, and swapped without rebuilding the rest.

It comes from a real need: agent governance offered by commercial platforms is expensive and ties you to one ecosystem, while the open source community has solid standalone pieces but not the coordinating layer that brings them together.

Context and problem

Organizations are deploying more and more AI agents, both in-house and third-party, and need to govern them the way they govern human identities: registry, identity, permissions, security, activity analytics, compliance and lifecycle.

The commercial market. Commercial agent governance platforms solve much of the problem, but with high costs, per-user licensing and dependence on a specific ecosystem. They tend to govern agents on their own platform well and those outside their perimeter poorly.

What even they lack. None fully covers a governed skills repository that is portable across platforms, resource control for local inference, or a harness declared per agent.

The open source gap. Quality open pieces exist: Microsoft Agent Governance Toolkit (policy engine, execution rings, kill switch, compliance), Galileo Agent Control (centralized policies), MCP gateways. What is missing is the coordinating layer that ties them together under a single inventory, governance model and UI. That is the gap this project fills.

Vision, mission and principles

Vision: any organization can govern its AI agents as rigorously as its human identities, without depending on a single vendor.

Mission: provide a minimal, stable governance core and an ecosystem of interchangeable modules that connect the best tools at each layer.

Design principles:

- **Modular and vendor-agnostic.** Each capability is a module with a defined contract; it does not matter whether an OSS project, a commercial product or custom code sits behind it.
- **Small core.** The core holds only what cannot be delegated: inventory, agent identity, policy model, audit and kill switch.
- **Deny by default.** If the policy plane does not respond, the action is not executed.
- **Self-hosted and air-gap capable.** No mandatory cloud dependency or outbound telemetry.
- **Every agent has an owner.** No agent runs in production without a human owner and a validated use case.
- **Standards over custom formats.** OpenTelemetry for traces, OPA/Rego or Cedar for policies, MCP for tools, OIDC/SPIFFE for identity.
- **Verifiable evidence.** Every policy decision and agent action leaves an immutable, auditable trail.

Scope

In scope:

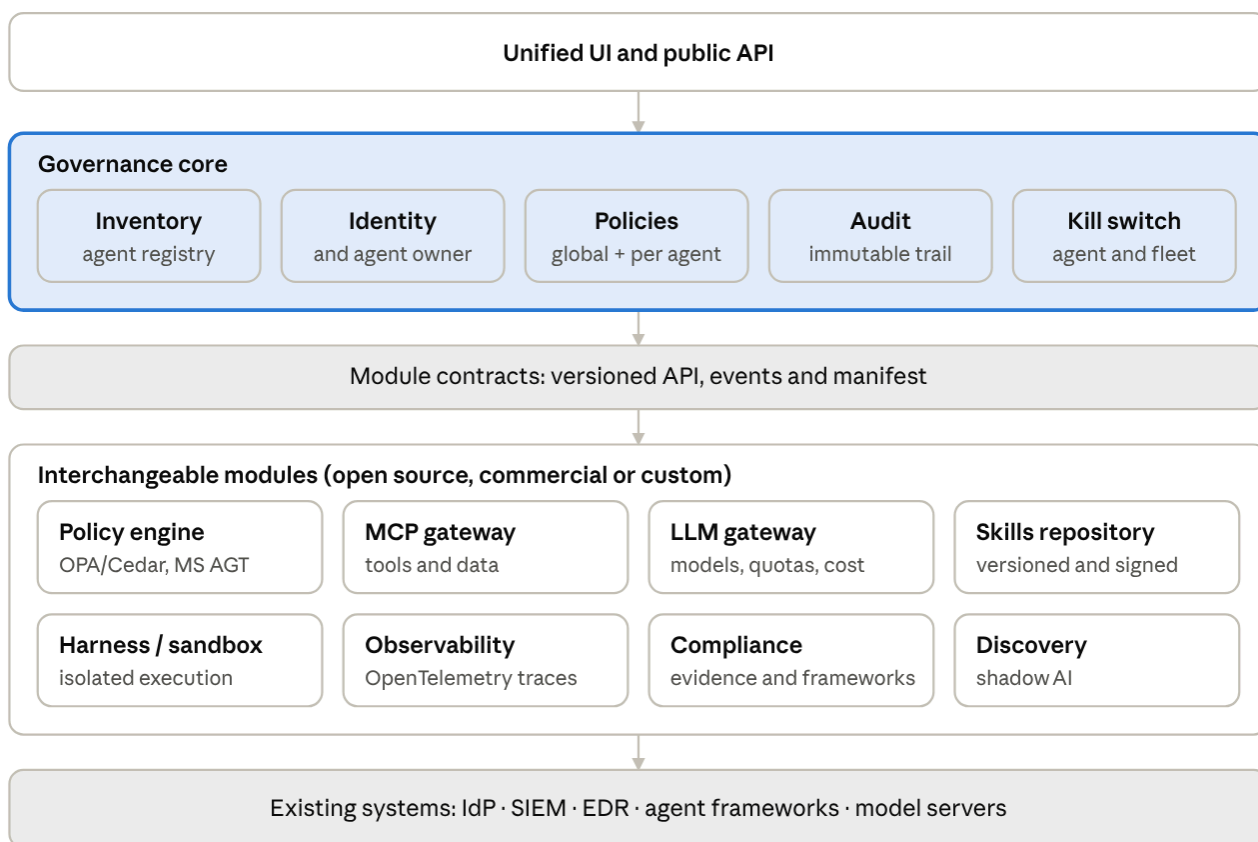
- Registry and inventory of in-house and third-party agents, including discovery of unregistered agents (shadow AI).
- Agent identity, agentic permissions and short-lived per-task credentials.
- Two-level policy engine: organization-wide and agent-specific.
- Deployment, monitoring and kill switch (per agent and fleet-wide).
- Resource control: token quotas, cost, GPU and local inference.
- Centralized, versioned skills repository with review and signing.
- Per-agent execution harness (sandbox).
- Observability, audit and compliance evidence.
- Unified UI and public API.
- SDK and contract specification so third parties can build modules.

Out of scope:

- Building a new agent framework (LangChain, CrewAI, OpenClaw, etc. are integrated).
- Serving models (vLLM, llama.cpp, Foundry Local or external APIs are integrated).
- Replacing the organization's IdP, SIEM or EDR: the project integrates with them.
- Reimplementing what an existing tool already does well: it is wrapped as a module.

Modular architecture

A small core governs; modules do the specialized work



architecture · core, contracts and 8 module types

The core only holds the shared truth: which agents exist, who is accountable for each, which policies apply to them, what they have done and how to stop them. Everything else is a module.

Module contract. Each module declares in a manifest which capability it covers, which events it emits and consumes, and which endpoints it exposes. An adapter translates the product's API (for example, Microsoft AGT or an MCP gateway) into the common contract. Switching tools means switching adapters.

Path of an agent action. The agent requests an action; the relevant gateway queries the policy engine with the agent’s identity and its specific policy; the decision and the action are recorded in the audit log. If the core does not respond, the action is denied.

Industry benchmarks

No single leader covers everything: each benchmark excels at one layer, and the project takes the best of each as its model instead of copying a single product.

Vendor and product	Focus	Recognition	What to take as inspiration
Zenity	Security and governance for agents across SaaS, cloud and endpoint	Named "Company to Beat" in AI agent governance by Gartner (April 2026) and a Market Shaper in AI application security (September 2026)	Shadow AI discovery, intent-aware runtime detection, MCP and A2A coverage
ServiceNow AI Control Tower	Cross-platform governance plane: discover, observe, govern, secure and measure	Positions itself as a governance layer regardless of where an agent is built or runs	Pre-approval of MCP servers, skill-level controls, role deviation detection, automatic permission rescoping, real-time shutdown, spend control
Microsoft Agent 365	Control plane built into the Microsoft ecosystem	Generally available since May 2026	Unified registry, agent identity and an accountable sponsor per agent
Credo AI	AI governance, risk and compliance	Leader in The Forrester Wave: AI Governance Solutions, Q3 2025	Policy management and regulatory mapping

Vendor and product	Focus	Recognition	What to take as inspiration
IBM watsonx.governance	AI lifecycle governance	Leader in the same Forrester Wave, Q3 2025	Evaluation, observability, guardrails and lifecycle
Google Gemini Enterprise Agent Platform and AWS Bedrock AgentCore	Governance built into the cloud platform	Cited as the hyperscaler alternatives (Forbes)	Google: cryptographic identity per agent, Agent Gateway and Agent Registry. AWS: harnesses and tool management
Kosmoy and Airia	Self-hosted or gateway-based control planes	Featured in industry comparisons	Kosmoy: kernel-level sandbox with kill switch. Airia: budgets and model routing

Conclusion. Zenity is the reference for runtime security and discovery; ServiceNow for cross-platform governance workflows and MCP and skill controls; Agent 365 and Google for agent identity; Credo AI and IBM for risk and compliance. The Forrester recognitions for Credo AI and IBM date from 2025 and cover AI governance in general, not agents specifically.

Capabilities and commercial references

Commercial products set the functional bar; the project replicates the essentials and improves where they fall short: ecosystem lock-in, per-user cost and agents outside their perimeter.

Capability	Commercial references	OSS candidate	What to improve
Unified registry and inventory	Agent 365 (central registry); ServiceNow (discovery across 30+ integrations, managed and unmanaged assets); Kosmoy (multi-cloud master registry)	Custom (core)	Register any agent without a specific vendor's SDK

Capability	Commercial references	OSS candidate	What to improve
Agent identity and permissions	Agent 365 + Entra Agent ID; Google (cryptographic identity per agent); ServiceNow with Veza (access graph, least privilege, automatic rescoping)	SPIFFE/SPIRE, Keycloak, MS AGT (DID identities)	Per-task credentials federated with the existing IdP
Runtime policies and detection	Zenity (intent-aware detection); ServiceNow (role deviation, output screening)	MS AGT (Agent OS), Galileo Agent Control , OPA	Global and per-agent policies in one model
Kill switch	ServiceNow (real-time shutdown); Kosmoy (Mission Control)	MS AGT (Agent Runtime)	Per-agent and fleet-wide stop, also outside the cloud
Sandbox / harness	Kosmoy (Action Capsule); AWS Bedrock AgentCore	gVisor, Firecracker, containers with seccomp	Harness declared per agent according to its use case
Resource and cost control	Airia (budgets, model routing); ServiceNow (spend dashboards)	LiteLLM	Include local inference (GPU, concurrency)
MCP and tool gateway	ServiceNow (MCP server pre-approval); Zenity (MCP and A2A); Google (Agent Gateway)	Obot, open MCP gateways	Tool permissions tied to the agent's policy
Skills repository	ServiceNow (provider and model approval at Skill Kit level)	Custom (Git + signing)	Open, portable repository with review, signing and versioning
Shadow AI discovery	Zenity; Agent 365	Integration with existing EDR/SIEM	Discovery without requiring a vendor's stack
Risk, compliance and evidence	Credo AI; IBM watsonx.governance	MS AGT (Agent Compliance)	Evidence exportable to GRC tools

The skills repository remains the clearest gap: ServiceNow controls which models skills can use inside its platform, but nobody offers an open repository that is portable across platforms. It is the project's natural differentiator.

Governance model

The platform turns the organization's governance model into software: no agent reaches production without an owner, a validated use case and a specific policy.

Role	Responsibility on the platform
AI steering committee	Approves global policies and accepted providers and jurisdictions
AI advisory committee (incl. security)	Validates use cases and reviews each agent's specific policy
Agent owner	Accountable for the agent's behavior, resources and periodic reviews
User	Uses authorized agents within the scope their policy allows
Platform operator	Runs modules and integrations; can trigger the kill switch

Agent lifecycle: proposal → use case validation → policy and harness assignment → deployment → monitored operation → periodic review → retirement. An agent without an active owner or with an expired review is suspended automatically.

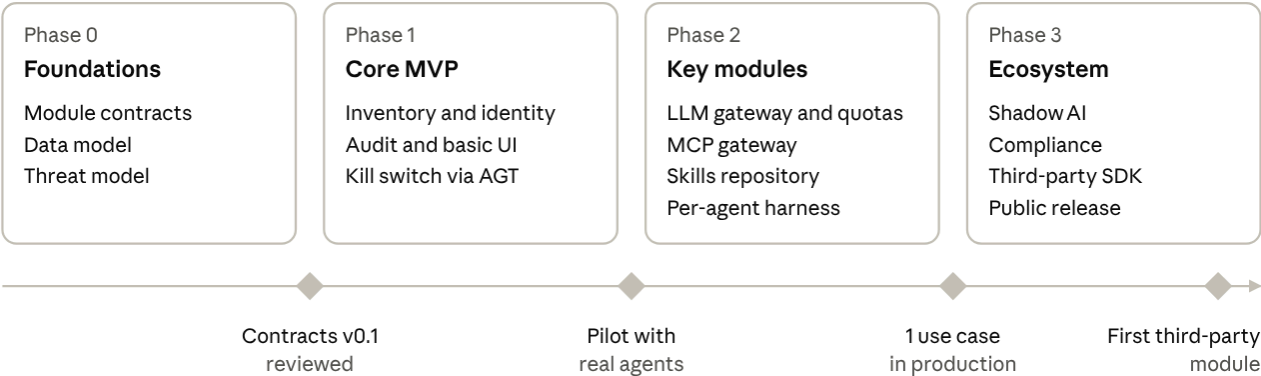
Non-functional and security requirements

- **Fail-closed:** if the policy engine or the core goes down, actions are denied, never allowed through.
- **Latency:** the policy decision must not become the bottleneck of the agent's action; target to be set in phase 0.
- **Kill switch:** must stop a running agent, not just revoke its next token; propagation time target to be set in phase 0.
- **Self-hosted:** deployed on containers and Kubernetes, with no cloud dependency; suitable for environments without Internet access.
- **Immutable audit:** signed logs or WORM storage, exportable to the organization's SIEM.

- **Existing identity integration:** OIDC/SAML with the organization's IdP; RBAC for human roles.
- **Security of the project itself:** published threat model, signed dependencies, SBOM, coordinated vulnerability disclosure.
- **Framework mapping:** OWASP Top 10 for Agentic AI and EU AI Act obligations.

Roadmap and success criteria

Four phases, each closed by a verifiable gate



roadmap · 4 phases and their gates, undated

Phases are not dated yet; dates will be set once the available team is known. Phase 1 is already useful on its own with a single adapter (Microsoft AGT).

Project success criteria:

- Every deployed agent appears in the inventory with an assigned owner and policy.
- The kill switch stops a running agent within the target time set in phase 0.
- At least one module is replaced by another vendor's module without changes to the core.
- Audit evidence is consumed from an external SIEM.

Risks, assumptions and project governance

Risk	Mitigation
OSS components change their APIs or are abandoned	Adapters isolated behind the contract; never couple the core to a tool
OSS projects acquired by commercial platforms	Our own permissive license and a preference for foundation-hosted projects

Risk	Mitigation
Scope creep: trying to build every capability	The core is limited to five functions; everything else is integrated
Lack of community adoption	Solve real use cases first and publish with documentation and examples
Community unable to sustain maintenance	Open governance from day one and defined maintainer roles

Assumptions: there is an initial community of contributors able to build the core; Microsoft AGT remains maintained and moves to a foundation.

Proposed license and governance: Apache 2.0 (permissive, with a patent grant), maintainer-led governance with an RFC process for changes to module contracts, and a medium-term goal of hosting the project in a foundation.

Sources

- [Zenity – Gartner recognitions](#)
- [Zenity named a Market Shaper in Gartner’s Emerging Market Quadrant](#)
- [ServiceNow expands AI Control Tower – ServiceNow Newsroom](#)
- [AI Control Tower: what’s new in the June 2026 release – ServiceNow Community](#)
- [ServiceNow Knowledge 2026 – The Letter Two](#)
- [Microsoft Agent 365 overview – Microsoft Learn](#)
- [Credo AI, Leader in The Forrester Wave: AI Governance](#)
- [IBM watsonx.governance, Leader in The Forrester Wave](#)
- [Microsoft makes governance the gate \(Google and AWS\) – Forbes](#)
- [Agent Governance Toolkit – Microsoft Open Source](#)
- [Agent Control – Galileo](#)
- [Top AI control plane tools – AIMultiple](#)
- [AI agent governance platforms 2026 – Kosmoy](#)